Simple Teaching Model (Mini E-Commerce)

We'll use only **4 collections**:

- User
- Product
- Order
- Category

Schema Design Overview

Relationships

Collection	Type Used	Why
User → Order	Reference	Large + reusable
Product → Category	Reference	Shared data
Order → Products	Snapshot + Reference	Price freeze
User → Address	Embedded	Small + fixed

Schemas

User Schema (Embedded + Methods + Statics)

```
const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String,

  address: { // Embedded
    city: String,
    pincode: String
  }
}, { timestamps: true });

/* METHOD */
userSchema.methods.generateToken = function () {
  return jwt.sign({ id: this._id }, "secret");
};

/* STATIC */
userSchema.statics.findByEmail = function (email) {
  return this.findOne({ email });
};
```

Category Schema (Simple Reference Target)

```
const categorySchema = new mongoose.Schema({
  name: String
});
```

Product Schema (Reference)

```
const productSchema = new mongoose.Schema({
  name: String,
  price: Number,

  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Category"
  }
});
```

Note:

- Product doesn't embed category → because category is reused

Order Schema (MOST IMPORTANT)

```
const orderSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  },

  products: [
    {
      productId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Product"
      },

      // SNAPSHOT
      name: String,
      price: Number,

      quantity: Number
    }
  ],

  totalAmount: Number
```

```
}, { timestamps: true });
```

Explain 3 Concepts Clearly

Embedded

```
address: { city, pincode }
Small + always used with user
```

Reference

```
user: ObjectId
category: ObjectId
Large + reusable
```

Snapshot (CRITICAL)

```
name: String,
price: Number
Stored inside order
```

Note:

"Even if product price changes tomorrow, order history must not change"

Populate (MongoDB JOIN)

Example

```
const orders = await Order.find()
  .populate("user")
  .populate("products.productId");
```

Note:

Without populate:

```
"user": "67abc123"
With populate:
"user": {
  "_id": "67abc123",
  "name": "Dinesh"
}
```

Note:

Populate replaces ObjectId with actual data

Aggregation (Advanced Query)

Example: Total Sales

```
const result = await Order.aggregate([
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$totalAmount" }
    }
  }
]);
```

Example: Category-wise Sales

```
const result = await Order.aggregate([
  { $unwind: "$products" },

  {
    $lookup: {
      from: "products",
      localField: "products.productId",
      foreignField: "_id",
      as: "productDetails"
    }
  },

  { $unwind: "$productDetails" },

  {
    $group: {
      _id: "$productDetails.category",
      total: { $sum: "$products.quantity" }
    }
  }
]);
```

Difference

Populate	Aggregation
Simple JOIN	Complex pipeline
Easy to use	Powerful
Limited control	Full control

Methods vs Statics (In Same Example)

Method

```
userSchema.methods.getPublicProfile = function () {
  return {
    name: this.name,
    email: this.email
  };
};
```

Static

```
userSchema.statics.login = async function (email, password) {
  const user = await this.findOne({ email });
  return user;
};
```

Flow

```
const user = await User.login(email, password); // static
const profile = user.getPublicProfile();        // method
```

End-to-End Flow

Place Order Logic

```
const cartProducts = await Product.find({ _id: { $in: ids } });

const orderProducts = cartProducts.map(p => ({
  productId: p._id,
  name: p.name,       // snapshot
  price: p.price,     // snapshot
  quantity: 1
}));

await Order.create({
  user: userId,
  products: orderProducts,
  totalAmount: 1000
});
```

Data Modeling Flow:

Step 1

Question:

"If product price changes, should old orders change?"

Step 2

Introduce:

- Embedded
- Reference
- Snapshot

Step 3

Show:

- .populate()

Step 4

Show:

- .aggregate()

Step 5

Add:

- Methods & statics

Final One-Line Summary

- **Embedded → store inside**
- **Reference → store id**
- **Snapshot → freeze data**
- **Populate → simple join**
- **Aggregation → powerful analysis**
- **Statics → fetch**
- **Methods → act**